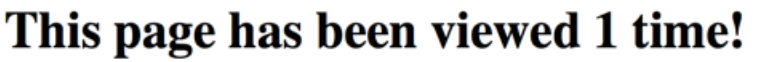
```
Rails.application.routes.draw do
  get 'welcome', to: 'welcome#index'
end
```

Now let's visit our Rails app in the browser at http://localhost:3000/welcome. You should see a page with our rendered welcome index.html.erb file, as shown in the following figure:



The page loads without errors—a good sign. Now try reloading the page. Every time you do, you should see the number of page hits increasing.

What does this mean? It means that our Rails app connected to the Redis server, incremented the value of "page hits" from default of 0 to 1, and finally displayed our welcome message with the number of page hits. More generally, we successfully got two containers to talk to each other. This is possible thanks to Compose creating the network for the app and automatically attaching containers to it.

Starting the Entire App with Docker Compose

We've just added Redis as a new service to our Compose file and configured our Rails app to talk to it. As we did this, the Rails server was already running, so we started the Redis server by itself with docker-compose run redis. However, one of the beauties of Compose is that no matter how many services we add to our application, we can manage it, in its entirety, with a single command, replacing the need for gems like Foreman.[3]

We can stop both the Rails server and Redis server in one go with:

```
$ docker-compose stop
```

You can verify that both services are stopped by running:

```
$ docker-compose ps
```

You should see something like this:

```
Name                   Command               State    Ports
------------------------------------------------------------------
myapp_redis_1   docker-entrypoint.sh redis ...   Exit 0
myapp_web_1     bin/rails s -b 0.0.0.0           Exit 1
```

3. https://rubygems.org/gems/foreman